🐶 Kerberos

gKerberos est un protocole qui permet à des principals (utilisateurs, postes, serveurs...) de s’authentifier de manière centralisée, et d’accéder à des services de manière authentifiée sans que le principal ait besoin d'envoyer son password et sans que le serveur ait besoin de connaitre le password de tout le monde.

L’authentification d’un client auprès d’un serveur va s’opérer en 3 échanges auprès de 3 services distincts :

1. **Authentication Service Exchange (AS exchange)** : Le client s'authentifie en faisant une demande de TGT auprès du KDC
2. **Ticket-Granting Service Exchange (TGS exchange)** : Le client demande un ST au TGS ce qui permettera d'accéder à un service choisi
3. **Client/Server Authentication Exchange (AP exchange)** : Le client communique avec le service en lui fournissant le ST

Chaque étape est composée d’une requête REQ et d’une réponse REP,
ce qui fait 6 échanges pour réaliser une authentification complète :

1. **KRB_AS_REQ** et **KRB_AS_REP**
2. **KRB_TGS_REQ** et **KRB_TGS_REP**
3. **KRB_AP_REQ** et **KRB_AP_REP**

Pour imager le fonctionnement global du protocole Kerberos avec la consommation d'alcool dans un festival organisé par la mairie :

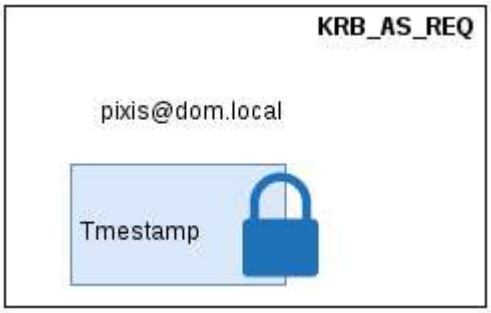
- Pour demander de l'alcool je dois avoir ma carte d'identité (TGT), pour se faire je l'a demande à la mairie (au KDC)
- Pour demander un vodka redbull (le service souhaité) je dois demander un ticket de consommation (ST) à la caisse de la mairie (au KDC qui joue le role de TGS)
- Je peux présenter mon ticket (ST) au bar de la mairie (au service souhaité) pour boire mon verre (communiquer avec le service en étant authentifié)
- Si maintenant je souhaite un verre de jack, il suffit juste de demander un autre ticket de consommation (ST) à la caisse de la mairie (au KDC qui joue le role de TGS) grace à ma carte d'identité précédemment demandée (TGT)

Authentication Service Exchange (AS exchange)

KRB_AS_REQ

Le client s'authentifie en faisant une demande de TGT auprès du KDC en envoyant son username, le nom du domaine AD, un timestamp (heure précise de la demande) chiffré avec son

hash NT et d'autres informations détaillés en dessous :

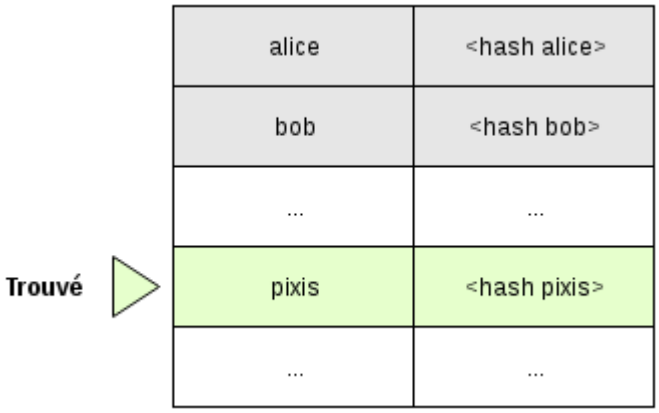


Détails d'un message KRB_AS_REQ :

```
[AS-REQ]
 pvno : numéro de version, toujours 5
 msg-type : type de message, 10 pour KRB_AS_REQ
 *padata: Séquence de blocs PA-DATA (voir ci-dessous)
   [PA-ENC-TIMESTAMP] : EncryptedData[PA-ENC-TS-ENC]
     [PA-ENC-TS-ENC] == Partie chiffrée par Kc ==
     patimestamp : 20140327121101
 req-body:
   [KDC-REQ-BODY]
     kdc-options: 0x40810010 - Options du ticket demandées par le client
       (enc_pa_rep, renewable, forwardable)
     *cname: Administrateur (1) - Nom du client
     realm: DEMOTEST
     *sname: krbtgt/DEMO.TEST (2) - Nom du service demandé
       (toujours krbtgt/REALM pour les TGT)
     till: 2037/09/13 02:48:05.0
     *rtime: 2037/09/13 02:48:05.0
     nonce: 0x5ec10613
     etype: Séquences des identifiants des algorithmes de chiffrement
            supportés par le client
       → 0x18 (aes256-cts-hmac-sha1-96)
       → 0x17 (aes128-cts-hmac-sha1-96)
       → 0x23 (rc4-hmac)
       → 0x24 (rc4-hmac-exp)
       → 0xffffff79 (rc4-hmac-old-exp)
       → 0x3 (des-cbc-md5)
     *addresses: Séquence des adresses du client
       → addr-type: 20 (NetBios), address: WIN8
```

KRB_AS_REP

Le KDC va vérifier alors que l'username existe bien dans NTDS.dit et tenter de déchiffrer le timestamp avec son hash NT pour vérifier que le client a utilisé le bon secret :



Si il arrive à déchiffrer le timestamp envoyé par le client, le KDC génère une clé de session unique, chiffré avec le hash NT de l'utilisateur et limité dans le temps.

Le KDC renvoie au client une réponse KRB_AS_REP qui contient :

- la clé de session généré par le KDC
- le TGT chiffré avec la LT key du KDC (le hash NT du compte krbtgt), qui contient :
  - L'username
  - La période de validité du ticket (par défaut 8 heures)
  - La clé de session généré
  - Le 🐶 Kerberos > PAC (Privilege Attribute Certificate) qui contient beaucoup d'informations sur l'utilisateur

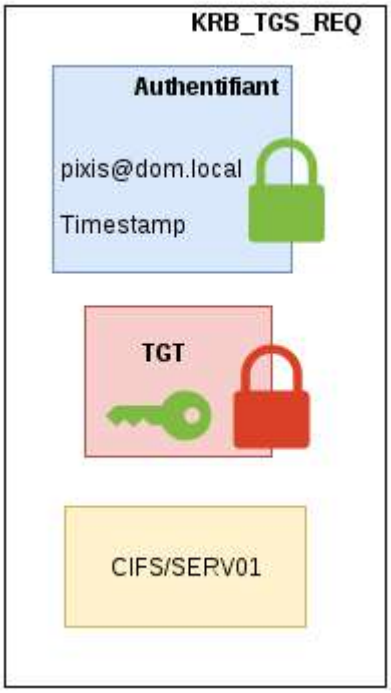
Détails d'un message KRB_AS_REP :

```
[AS-REP]
  pvno : numéro de version, toujours 5
  msg-type : type de message, 11 pour KRB_AS_REP
  *padata: Séquence de blocs PA-DATA
    [PA-ETYPE-INFO2]
      etype: 0x18 (aes256-cts-hmac-sha1-96)
      salt: DEMO.TESTAdministrateur
  crealm: DEMO.TEST
  cname: Administrateur (1)
  ticket:
    [Ticket] TGT transmis au client
      tkt-vno: numéro de version, toujours 5
      realm: DEMO.TEST
      sname: krbtgt/DEMO.TEST (2)
      enc-part: EncryptedData[EncTicketPart] etype: 18, kvno: 2
        [EncTicketPart] == Partie chiffrée par K_KDC ==
          flags: 0x40e10000
           (enc_pa_rep, pre_authent, initial, renewable, forwardable)
          key: [EncryptionKey] keytype: 0x12, keyvalue:
            Clé de session S_C,K contenue dans le ticket
          crealm: DEMO.TEST
          cname: Administrateur (1)
          authtime: 2014/03/27 11:11:58.0
          *starttime: 2014/03/27 11:11:58.0
          endtime: 2014/03/27 21:11:58.0
          *renew-till: 2014/04/03 11:11:58.0
          *authorization-data: cf. partie 2.5
  enc-part: EncryptedData[EncASRepPart] etype: 23, kvno: 1
    [EncKDCRepPart] == Partie chiffrée par K_C ==
      key: [EncryptionKey] keytype: 0x12, keyvalue:
        Clé de session S_C,K transmise au client
      nonce 0x5ec10613 - aléa repris de la requête du client
      *key-expiration: 2037/09/14 02:48:05.0
      flags: 0x40e10000
        (enc_pa_rep, pre_authent, initial, renewable, forwardable)
      authtime: 2014/03/27 11:11:58.0
      starttime: 2014/03/27 11:11:58.0
      endtime: 2014/03/27 21:11:58.0
      *renew-till: 2014/04/03 11:11:58.0
      srealm: DEMO.TEST
      sname: krbtgt/DEMO.TEST (2)
      *caddr:
        → 0x18 addr-type: 20 (NetBios), address: WIN8
```

KRB_TGS_REQ

Maintenant que le client a pu s'authentifié, si le client veut utiliser un service il va faire une demande de ST au TGS, ici le KDC, en envoyant :

- le TGT
- le SPN du service souhaité
- un Authenticator, un message contenant son username et un timestamp, le tout chiffré avec la clé de session



Détails d'un message KRB_TGS_REQ :

```
[TGS-REQ]
  pvno : numéro de version, toujours 5
  msg-type : type de message, 12 pour KRB_TGS_REQ
  *padata: Séquence de blocs PA-DATA
    [AP-REQ] - Message KRB_AP_REQ authentifiant le client auprès du service TGS
      pvno: 5
      msg-type: 14
      ap-option: 0x0 (pas d'option)
      ticket:
        [Ticket] TGT transmis du client au KDC (extraits, voir AS-REP)
          ...
          sname: krbtgt/DEMO.TEST (2)
          enc-part: EncryptedData[EncTicketPart] etype: 18, kvno: 2
            [EncTicketPart] == Partie chiffrée par K_KDC ==
              ...
              key: [EncryptionKey] keytype: 0x12, keyvalue:
                Clé de session S_C,K contenue dans le ticket
              ...
      authenticator: EncryptedData[Authenticator] etype: 18
        [Authenticator] == Partie chiffrée par S_C,K ==
          authenticator-vno: 5
          crealm: DEMO.TEST
          cname: Administrateur (1)
          *cksum:
            [Checksum]
              cksumtype: 0x7
              checksum: f166b445ae30928f9e74874d60c94e83
          ctime: 2014/03/27 11:12:00.0
          *seq-number: 61eb3541
  req-body:
    [KDC-REQ-BODY]
      kdc-options: 0x40810000 - Options de ticket demandée par le client
        (enc_pa_rep, renewable, forwardable)
      realm: DEMO.TEST
      *sname: cifs/dc-2012-01.demo.test (2) - Nom du service demandé
      till: 2037/09/13 02:48:05.0
        Date de validité maximale proposée par le client pour le ticket
      nonce: 0x61eb3541
      etype: Séquences d'identifiant des algorithmes de chiffrement
             supportés par le client
        → 0x18 0x18 (aes256-cts-hmac-sha1-96)
        → 0x18 0x17 (aes128-cts-hmac-sha1-96)
        → 0x18 0x23 (rc4-hmac)
        → 0x18 0x24 (rc4-hmac-exp)
        → 0x18 0xffffff79 (rc4-hmac-old-exp)
      *enc-authorization-data: cf. partie 2.5
```

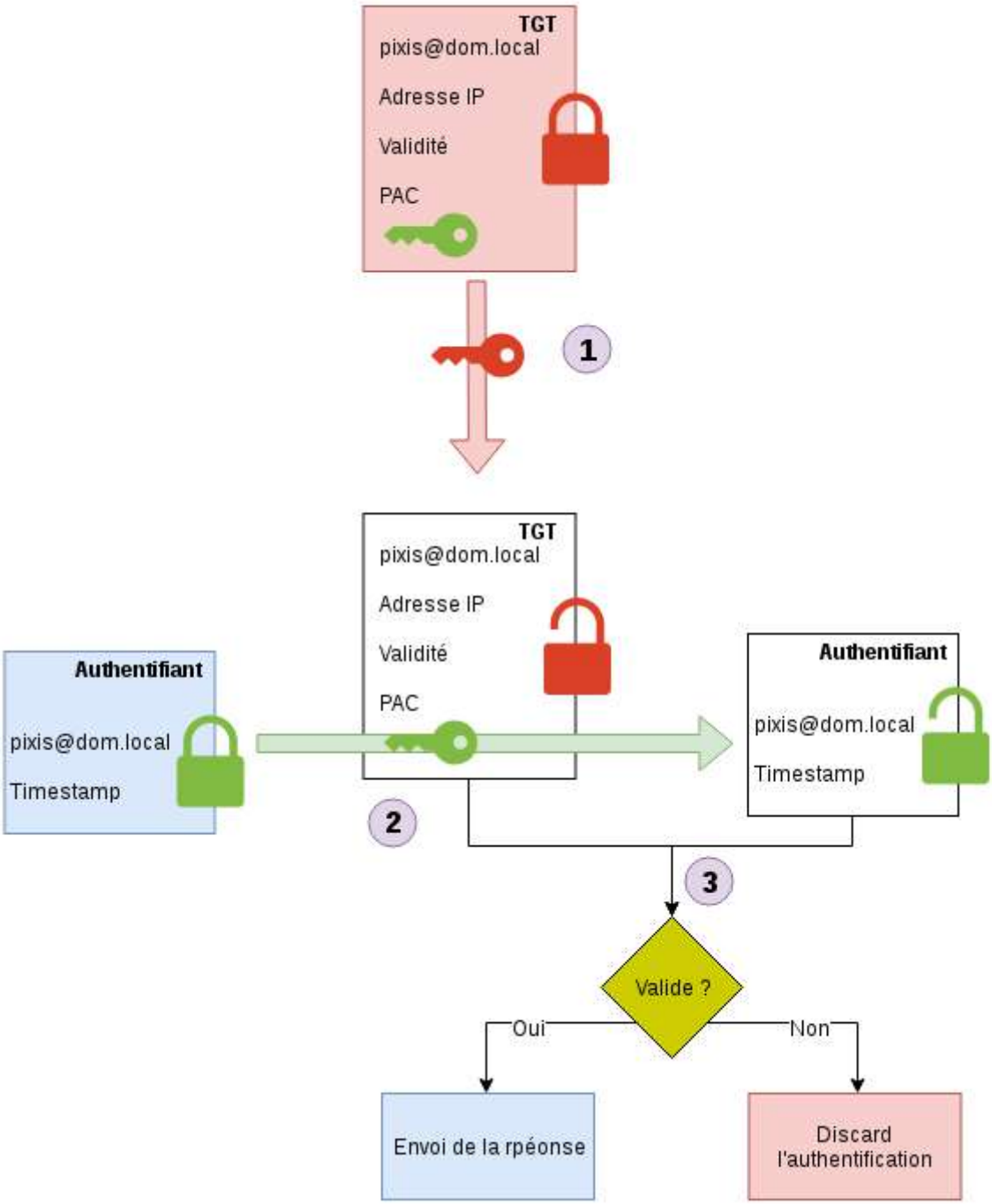
Ticket-Granting Service Exchange (TGS exchange)

KRB_TGS_REP

Le KDC va vérifier que c'est le bon utilisateur qui fait la demande en comparant le contenu du TGT (Comme le KDC est le seul à pouvoir lire le contenu du TGT) et de l'authenticator.

Il déchiffre le contenu de l'authenticator avec la clé de session récupéré dans le TGT.
Si le déchiffrement fonctionne => données dans l'authenticator = données dans le TGT.
Cela permet de s'assurer que le client possède le TGT et qu'il a connaissance de la clé de session.

Pour résumer la vérification du KDC :



1. Le KDC déchiffre le TGT avec sa clé
2. Le KDC la clé de session qui est dans le TGT pour déchiffrer l'authenticator
3. Le KDC compare le contenu

Il vérifie aussi le service souhaité par l'utilisateur.

Maintenant que le KDC a pu vérifier les informations, il va envoyé au client un ST permettant à l'utilisateur d'accéder au service voulu dans une réponse KRB_TGS_REP qui contient :

- un ST chiffré avec la LT key du service souhaité (le hash NT du compte de service), qui contient :
  - le SPN du service souhaité
  - l'username
  - le PAC
  - une nouvelle clé de session limité dans le temps valide uniquement pour l'utilisateur voulant communiquer avec le service
- La nouvelle clé de session

Ces deux informations (le ST et la nouvelle clé de session) sont chiffrées avec la première clé de session négocié lors de l'AS exchange

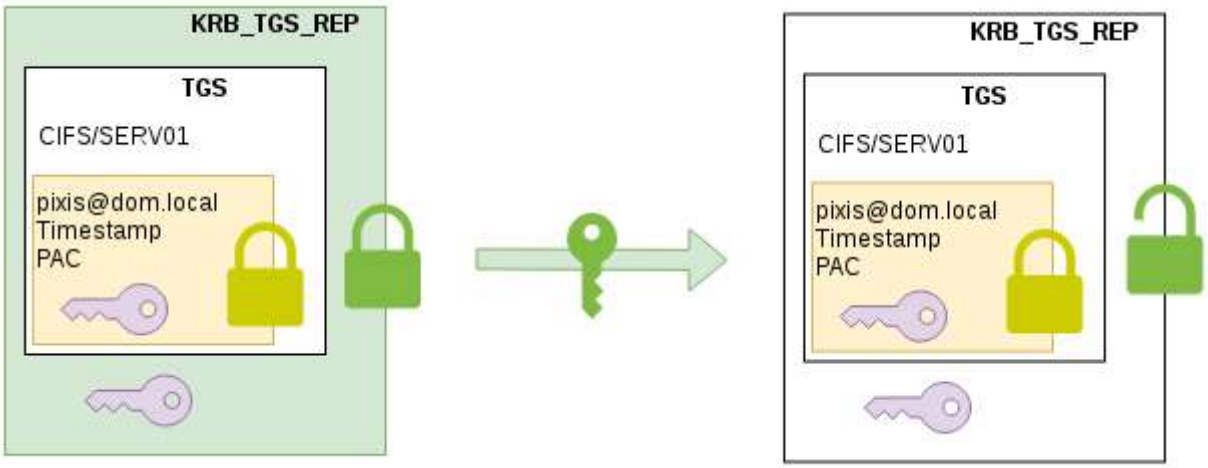
Détails d'un message KRB_TGS_REP :

```
[TGS-REP]
| pvno : numéro de version, toujours 5
| msg-type : type de message, 13 pour KRB_TGS_REP
| crealm: DEMO.TEST
| cname: Administrateur (1)
| ticket:
|   [Ticket] Ticket de service transmis au client
|   | tkt-vno: 5
|   | realm: DEMO.TEST
|   | sname: cifs/dc-2012-01.demo.test (2) - SPN du service
|   | enc-part: EncryptedData[EncTicketPart] etype: 18, kvno: 10
|   |   [EncTicketPart] == Partie chiffrée par K_S ==
|   |   || flags: 0x40a10000
|   |   ||   (enc_pa_rep, cname_in_pa_data, pre_authent, renewable, forwardable)
|   |   || key: [EncryptionKey] keytype: 0x12, keyvalue:
|   |   ||   Clé de session S_C,S contenue dans le ticket
|   |   || crealm: DEMO.TEST
|   |   || cname: Administrateur (1)
|   |   || authtime: 2014/03/27 11:11:58.0 - Date de validité du ticket
|   |   || *starttime: 2014/03/27 11:12:00.0
|   |   || endtime: 2014/03/27 21:11:58.0
|   |   || *renew-till: 2014/04/03 11:11:58.0
|   |   || *authorization-data: Données d'autorisation, cf. partie 2.5
| enc-part: EncryptedData[EncTGSRepPart] etype: 18
|   [EncKDCRepPart] == Partie chiffrée par S_C,K ==
|   || key: [EncryptionKey] keytype: 0x12, keyvalue:
|   ||   Clé de session S_C,S transmise au client
|   || nonce 0x61eb3541 - Aléa repris de la demande du client
|   || flags: 0x40a10000
|   ||   (enc_pa_rep, cname_in_pa_data, pre_authent, renewable, forwardable)
|   || authtime: 2014/03/27 11:11:58.0
|   || starttime: 2014/03/27 11:12:00.0
|   || endtime: 2014/03/27 21:11:58.0
|   || *renew-till: 2014/04/03 11:11:58.0
|   || srealm: DEMO.TEST
|   || sname: cifs/dc-2012-01.demo.test (2)
```

Client/Server Authentication Exchange (AP exchange)

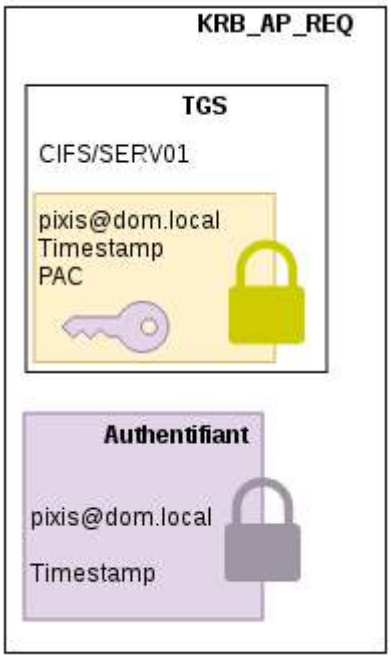
KRB_AP_REQ

Le client recoit le message KRB_TGS_REP et déchiffre le message avec la premiere clé de session pour récupérer le ST et la deuxieme clé de session pour la communication avec le service souhaité :



Maintenant que le client a récupéré ces informations, meme processus que lors d'une demande de ST dans un message KRB_TGS_REQ. Le client demande d'accéder au service en envoyant :

- le ST chiffré avec la clé du service
- un nouveau authenticator (username + timestamp chiffré) généré par l'utilisateur et chiffré avec la nouvelle clé de session

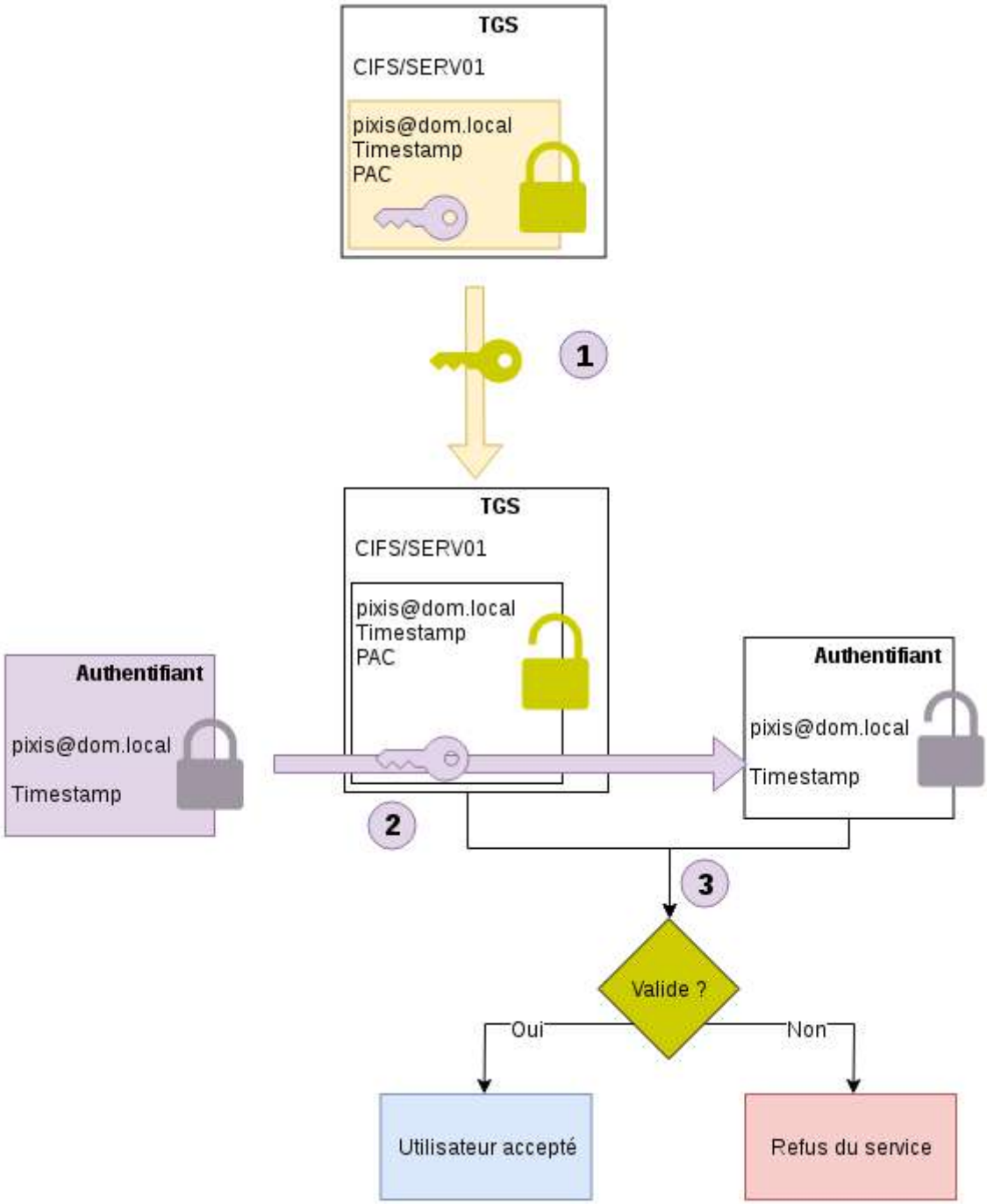


Détails d'un message KRB_AP_REQ :

```
[AP-REQ]
| pvno : numéro de version, toujours 5
| msg-type : type de message, 14 pour KRB_AP_REQ
| ap-option: 0x20000000 - Options du message AP-REQ demandées par le client
|   (mutual-required)
| ticket:
|   [Ticket] Ticket de service transmis du client au serveur
|   | ...
|   | sname: cifs/dc-2012-01.demo.test (2)
|   | enc-part: EncryptedData[EncTicketPart] etype: 18, kvno: 10
|   |   [EncTicketPart] == Partie chiffrée par K_S ==
|   |   || ...
|   |   || key: [EncryptionKey] keytype: 0x12, keyvalue:
|   |   ||   Clé de session S_C,S contenue dans le ticket
| authenticator: EncryptedData[Authenticator] etype: 18
|   [Authenticator] == Partie chiffrée par S_C,S ==
|   || authenticator-vno: 5
|   || crealm: DEMO.TEST
|   || cname: Administrateur (1) - Nom du client
|   || *cksum:
|   ||   [Checksum]
|   ||   | cksumtype: 0x7
|   ||   | checksum: 42ab124f520aee347b379039c490934
|   || ctime: 2014/03/27 11:12:00.0 - Temps actuel du client
|   || *subKey: [EncryptionKey] keytype: 0x12, keyvalue:
|   ||   Clé de session C->S supplémentaire
|   || *seq-number: 61ea6f95 - Numéro de séquence
|   || *authorization-data: Données d'autorisation, cf. partie 2.5
```

KRB_AP_REP

Le service déchiffre le ST avec sa clé de service pour récupérer la deuxieme clé de session afin de déchiffrer l'authenticator
Si le déchiffrement fonctionne => données dans l'authenticator = données dans le ST.



1. Le service déchiffre les informations contenu dans le ST avec sa clé de service.
2. Le service utilise la deuxieme clé de session pour déchiffrer l'authenticator
3. Le service compare le contenu

Maintenant que le service a vérifié ces informations, une authentification mutuelle va etre établit, le serveur qui héberge le service va prouver au client qu'il connait la clé de session en envoyant dans un message KRB_AP_REP :

- le timestamp de l'authenticator message chiffré avec la clé de session

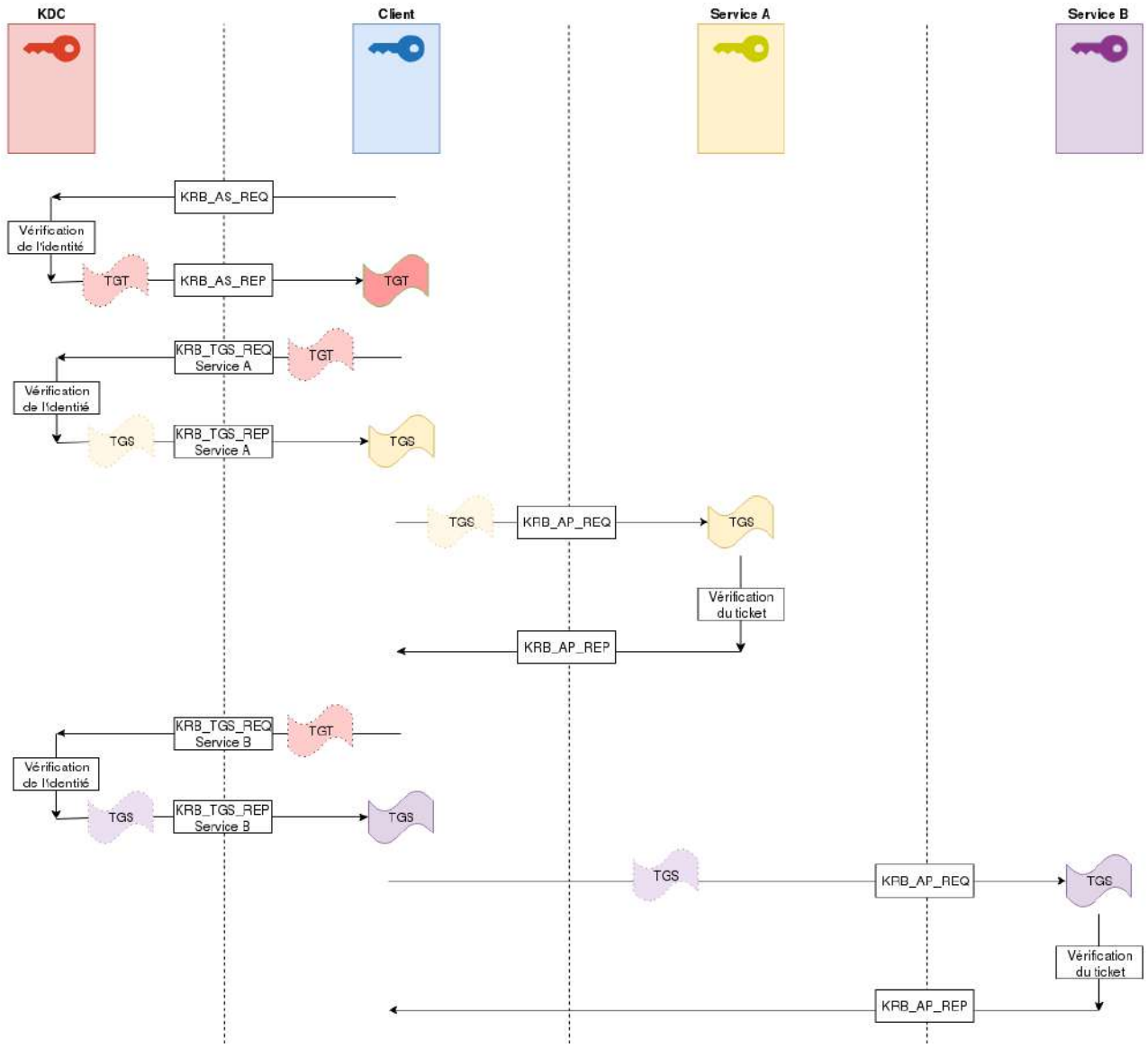
Ensuite le client recoit le message KRB_AP_REP et déchiffre avec la clé de session le timestamp envoyé et le compare avec le timestamp contenu dans son authenticator message original.
S'ils correspondent, le client est assuré que le service est authentique et la connexion se poursuit.

Pour plus de détails : https://learn.microsoft.com/en-us/windows/win32/secauthn/client-server-exchange

Détails d'un message KRB_AP_REP :

```
[AP-REP]
| pvno: numéro de version, toujours 5
| msg-type: type de message, 15 pour KRB_AP_REP
| enc-part: EncryptedData[EncAPRepPart] etype: 18
|   [EncAPRepPart] == Partie chiffrée par S_{C,S} ==
|   || ctime: 2014/03/27 11:12:00.0
|   || *subkey: [EncryptionKey] keytype: 0x12, keyvalue:
|   ||   Clé de session S<->C supplémentaire
|   || *seq-number: 0x61c8098c
```

Résumé : authentification + accès à deux services via Kerberos

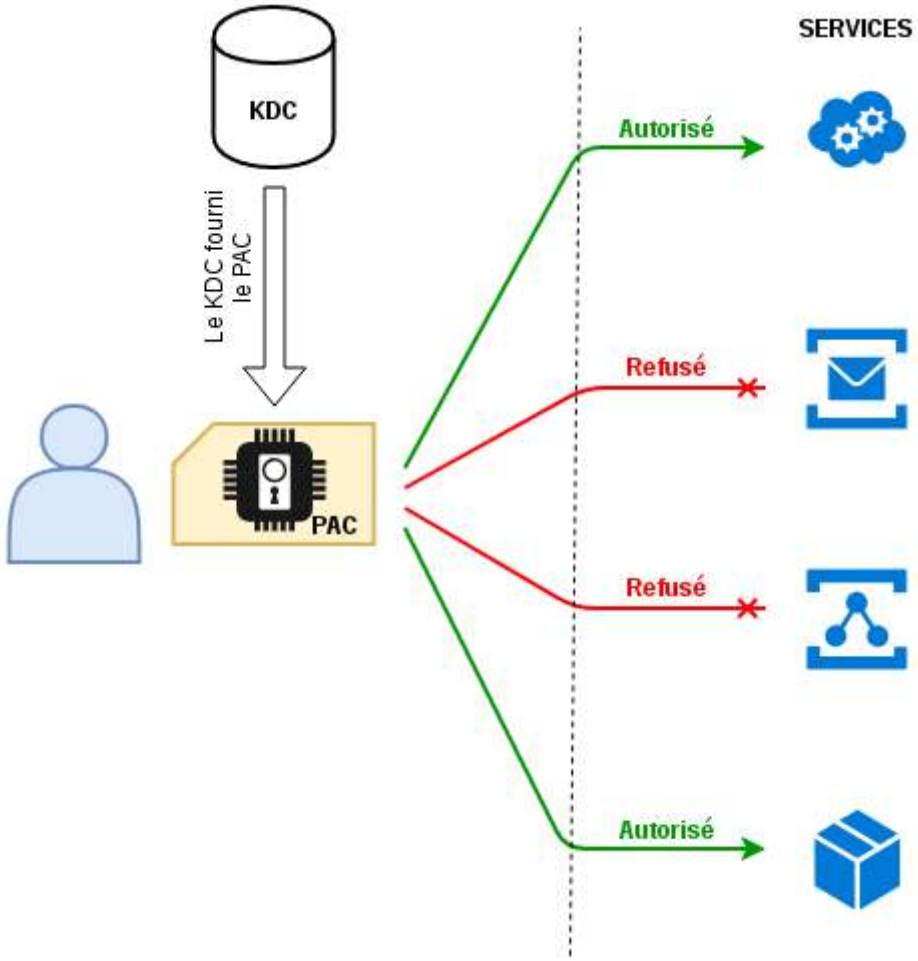


(Faute sur les schémas de pixis : TGS à la place de ST)

PAC (Privilege Attribute Certificate)

Le PAC est une extension du protocole Kerberos utilisé par Microsoft pour la gestion des droits dans un environnement Active Directory.
Seul le KDC connait les droits de chacun des objets, il est donc nécéssaire pour lui de transmettre ces informations aux différents services afin qu'ils puissent créer des access tokens adaptés aux utilisateurs qui utilisent ces services.

Le PAC est très important car cela permet à un utilisateur d'accéder ou non à un service, effectuer certaines actions etc... :



Le PAC peut être considéré comme le badge de sécurité de l'utilisateur : Il peut l'utiliser pour ouvrir des portes, mais il ne peut pas ouvrir des portes auxquelles il n'a pas accès.

Le PAC se trouve dans les TGT (le PAC est alors chiffré avec la clé du KDC) et dans les ST. (le PAC est alors chiffré avec la clé du service demandé)
L'utilisateur n'a donc pas la main sur ces informations, il ne peut pas modifier ses propres droits, groupes, etc.

Voici la structure du PAC :

```
typedef struct _KERB_VALIDATION_INFO {
   FILETIME LogonTime;
   FILETIME LogoffTime;
   FILETIME KickOffTime;
   FILETIME PasswordLastSet;
   FILETIME PasswordCanChange;
   FILETIME PasswordMustChange;
   RPC_UNICODE_STRING EffectiveName;
   RPC_UNICODE_STRING FullName;
   RPC_UNICODE_STRING LogonScript;
   RPC_UNICODE_STRING ProfilePath;
   RPC_UNICODE_STRING HomeDirectory;
   RPC_UNICODE_STRING HomeDirectoryDrive;
   USHORT LogonCount;
   USHORT BadPasswordCount;
   ULONG UserId;
```

```
    ULONG PrimaryGroupId;
    ULONG GroupCount;
    [size_is(GroupCount)] PGROUP_MEMBERSHIP GroupIds;
    ULONG UserFlags;
    USER_SESSION_KEY UserSessionKey;
    RPC_UNICODE_STRING LogonServer;
    RPC_UNICODE_STRING LogonDomainName;
    PISID LogonDomainId;
    ULONG Reserved1[2];
    ULONG UserAccountControl;
    ULONG SubAuthStatus;
    FILETIME LastSuccessfulILogon;
    FILETIME LastFailedILogon;
    ULONG FailedILogonCount;
    ULONG Reserved3;
    ULONG SidCount;
    [size_is(SidCount)] PKERB_SID_AND_ATTRIBUTES ExtraSids;
    PISID ResourceGroupDomainSid;
    ULONG ResourceGroupCount;
    [size_is(ResourceGroupCount)] PGROUP_MEMBERSHIP ResourceGroupIds;
 } KERB_VALIDATION_INFO, *PKERB_VALIDATION_INFO;
```

(https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-pac/69e86ccc-85e3-41b9-b514-7d969cd0ed73)

Voici un exemple de PAC d'un utilisateur du domaine dumpé avec getPac via une request S4USelf + U2U auth :

```
❯ getPac.py -targetUser Administrator cat.local/'Administrator':'Passw0rd!'
Impacket v0.9.24 - Copyright 2021 SecureAuth Corporation

KERB_VALIDATION_INFO
LogonTime:
    dwLowDateTime:                  568415828
    dwHighDateTime:                 31000949
LogoffTime:
    dwLowDateTime:                  4294967295
    dwHighDateTime:                 2147483647
KickOffTime:
    dwLowDateTime:                  4294967295
    dwHighDateTime:                 2147483647
PasswordLastSet:
    dwLowDateTime:                  2158304949
    dwHighDateTime:                 30995771
PasswordCanChange:
    dwLowDateTime:                  2869878453
    dwHighDateTime:                 30995972
PasswordMustChange:
    dwLowDateTime:                  4294967295
    dwHighDateTime:                 2147483647
EffectiveName:                  'Administrator'
FullName:                       ''
```

```
LogonScript:                    ''
ProfilePath:                    ''
HomeDirectory:                  ''
HomeDirectoryDrive:             ''
LogonCount:                     12
BadPasswordCount:               0
UserId:                         500
PrimaryGroupId:                 513
GroupCount:                     5
GroupIds:
    [

        RelativeId:                     520
        Attributes:                     7 ,

        RelativeId:                     512
        Attributes:                     7 ,

        RelativeId:                     513
        Attributes:                     7 ,

        RelativeId:                     518
        Attributes:                     7 ,

        RelativeId:                     519
        Attributes:                     7 ,
    ]
UserFlags:                      544
UserSessionKey:
    Data:
b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'
LogonServer:                    'DC01'
LogonDomainName:                'CAT'
LogonDomainId:
    Revision:                       1
    SubAuthorityCount:              4
    IdentifierAuthority:            b'\x00\x00\x00\x00\x00\x05'
    SubAuthority:
        [
             21,
             2083336711,
             2804761123,
             4253890002,
        ]
LMKey:                          b'\x00\x00\x00\x00\x00\x00\x00\x00'
UserAccountControl:             528
SubAuthStatus:                  0
LastSuccessfulILogon:
    dwLowDateTime:                  0
    dwHighDateTime:                 0
LastFailedILogon:
    dwLowDateTime:                  0
    dwHighDateTime:                 0
```

```
FailedILogonCount:                0
Reserved3:                        0
SidCount:                         1
ExtraSids:
    [

        Sid:
            Revision:                       1
            SubAuthorityCount:              1
            IdentifierAuthority:            b'\x00\x00\x00\x00\x00\x12'
            SubAuthority:
                [
                     2,
                ]
        Attributes:                     7 ,
    ]
ResourceGroupDomainSid:
    Revision:                       1
    SubAuthorityCount:              4
    IdentifierAuthority:            b'\x00\x00\x00\x00\x00\x05'
    SubAuthority:
        [
             21,
             2083336711,
             2804761123,
             4253890002,
        ]
ResourceGroupCount:               1
ResourceGroupIds:
    [

        RelativeId:                     572
        Attributes:                     536870919 ,
    ]
Domain SID: S-1-5-21-2083336711-2804761123-4253890002
```

http://remivernier.com/index.php/2018/07/07/kerberos-exploration/

Kerberos Service for User (S4U) Extensions

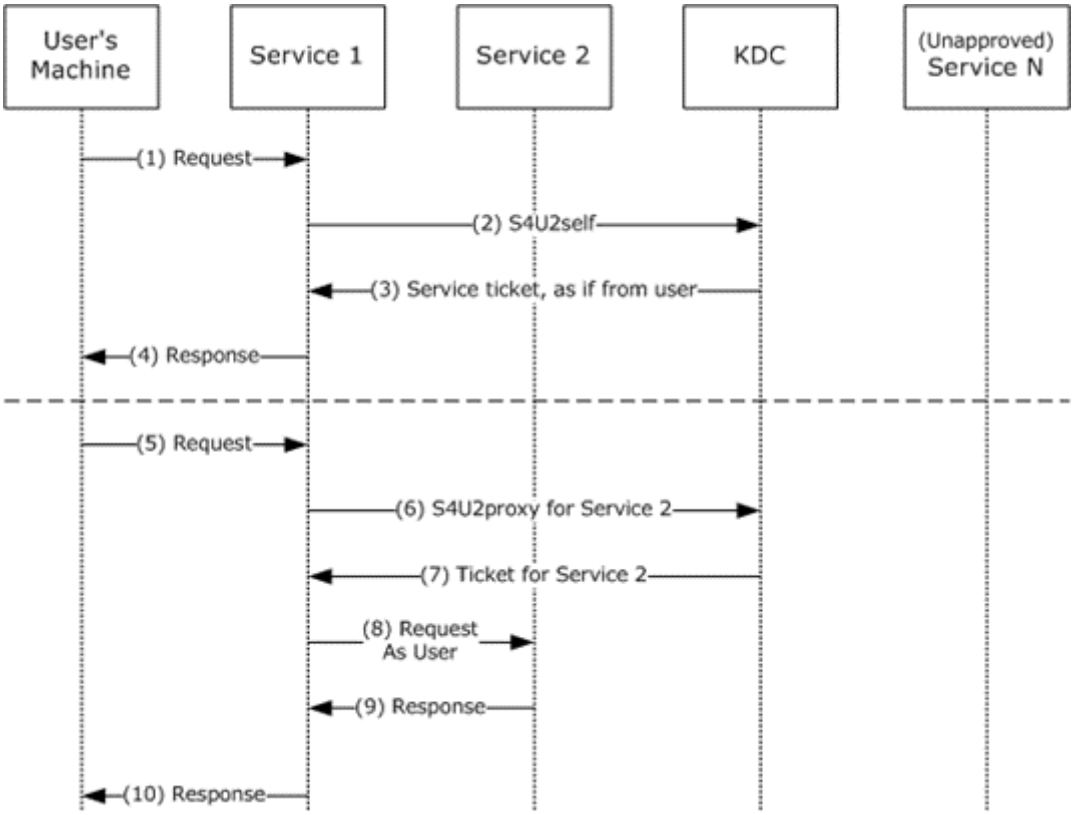
Les extensions S4U sont des extensions du protocole Kerberos qui permettent à un service d'obtenir un ST pour un utilisateur qui ne s'est pas authentifié auprès du KDC
S4U comprend : S4U2proxy et S4U2self

Service for User to Self (S4U2self)

Permet à un service d'obtenir un ST au nom d'un utilisateur, vers lui meme

Service for User to Proxy (S4U2proxy)

Permet à un service d'obtenir un ST au nom d'un utilisateur, vers un autre service



User-to-User (U2U) Authentication

Permet à un utilisateur de demander un ST pour un autre utilisateur

Glossary

- TGT (Ticket-Granting Ticket) : Le ticket qui est utilisé pour demander d'autres tickets. Il est obtenu lors du première échange d'une authentification Kerberos (AS exchange), il est dans la réponse KRB_AS_REP.
- ST (Service Ticket) : Le ticket permettant d'accéder à un service. Il est obtenu dans la réponse KRB_TGS_REP lors d'un TGS exchange.
- KDC (Key Distribution Center) : Le service centralisé qui gère l'authentification Kerberos dans un réseau. Dans un environnement Active Directory, le KDC est le DC.
- TGS (Ticket-Granting Service) : Le service qui émet les ST. (donc le KDC)
- LT key (Long Term key) : Une clé utilisé pour chiffrer les TGT ou les ST qui est enfaites rien de plus que si l'etype (Encryption Type) RC4 est pas désactivé : le hash NT du compte qui chiffre le ticket (TGT => krbtgt / ST => compte de service) sinon la clé Kerberos (DES, AES128 ou AES256) du compte qui chiffre le ticket. (TGT => krbtgt / ST => compte de service)
- S4U2self (Service for User to Self) : Extention Kerberos qui permet à un service d'obtenir un ST au nom d'un utilisateur, vers lui meme.
- S4U2proxy (Service for User to Proxy) : Extension Kerberos qui permet à un service d'obtenir un ST au nom d'un utilisateur, vers un autre service.
- U2U (User-to-User (U2U) Authentication) : Authentification qui permet à un utilisateur de demander un ST pour un autre utilisateur.

Ressources

https://beta.hackndo.com/kerberos/
https://youtu.be/5N242XcKAsM
https://www.thehacker.recipes/ad/movement/kerberos
https://www.sstic.org/media/SSTIC2014/SSTIC-actes/secrets_dauthentification_pisode_ii__kerberos_cont/SSTIC2014-Article-secrets_dauthentification_pisode_ii__kerberos_contre-attaque-bordes_2.pdf
https://www.ietf.org/rfc/rfc4120.txt
https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-sfu/4a624fb5-a078-4d30-8ad1-e9ab71e0bc47
https://web.mit.edu/rhel-doc/4/RH-DOCS/rhel-rg-fr-4/s1-kerberos-terminology.html
https://github.com/SecureAuthCorp/impacket/blob/master/examples/getPac.py
https://github.com/ShutdownRepo/impacket/blob/describeTicket/examples/describeTicket.py
https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-sfu/3bff5864-8135-400e-bdd9-33b552051d94
https://www.youtube.com/watch?v=7_iv_eaAFyQ
https://harmj0y.medium.com/s4u2pwnage-36efe1a2777c
https://shenaniganslabs.io/2019/01/28/Wagging-the-Dog.html
https://i.blackhat.com/USA-22/Wednesday/US-22-Forshaw-Taking-Kerberos-To-The-Next-Level.pdf
http://www.di-srv.unisa.it/~ads/corso-security/www/CORSO-0001/kerberos/ref/kerberos-faq.html